



User manual ioBroker.miboxer-wl433

Control MiBoxer pool lights (gateway WL-433, lamps PW01 / PW02) locally with ioBroker – explained step by step

Adapter version	0.1.0
Date	2026-09-22
Requirements	ioBroker with Admin 8 or newer, js-controller 6.0.11 or newer, Node.js 22 or newer
Language	English · Deutsche Fassung

Contents

- 1 About this manual
- 2 What the adapter does
- 3 What you need
- 4 Step 1 – Set up the lamps and the gateway in the MiBoxer app
- 5 Step 2 – Get the device ID and the local key
- 6 Step 3 – Install the adapter
- 7 Step 4 – Set up the instance
- 8 Step 5 – Check that everything works
- 9 Operating the pool lights
- 10 Zones
- 11 Automation examples
- 12 Troubleshooting
- 13 For the curious: how it works in detail
- 14 Glossary
- 15 Legal notice and contact

1. About this manual

This manual takes you from a MiBoxer app that is already set up to pool lights you switch, dim and colour from ioBroker. You **do not need any programming knowledge**. Every step is described so that you can simply work through it in order.

How to read this manual:

- Work through chapters 3 to 8 **in order**. After that, the control works.
- Chapters 9 to 11 show what you can do afterwards. Chapter 12 helps when something does not work.
- Numbered lists are steps to carry out: first step 1, then step 2 and so on.
- Words in `this font` are typed exactly like that or appear exactly like that on the screen.
- Chapter 14 explains unfamiliar terms.

Coloured boxes point out something special:

TIP

A tip makes your work easier.

NOTE

A note explains the background.

IMPORTANT

Important: do not overlook this.

CAUTION

Caution: something can go wrong here if you are not careful.

What has been checked: all steps with the adapter were tried out on 2026-09-22 with a real WL-433 gateway. The pictures come from ioBroker Admin 8. With another Admin version, buttons may look slightly different or have other names. Places that could not be tried out are marked **(unverified)**.

2. What the adapter does

The MiBoxer WL-433 gateway is the bridge between your home network (WiFi) and the pool lights. The lamps receive their commands by radio (LoRa, 433 MHz) from the gateway. Normally you control the gateway with the MiBoxer app – a lot of it goes through the internet (the "cloud" of the manufacturer Tuya).

The adapter **ioBroker.miboxer-wl433** talks to the gateway **directly in your home network**, without a detour through the internet:

```
ioBroker —(home network / WiFi)→ WL-433 gateway —(radio 433 MHz)→ PW01 / PW02 pool lights
```

What you can do with the adapter:

- Switch the lamps on and off and dim them (1–100 %)
- Set white light from warm (2700 K) to cold (6500 K)
- Set any colour, including pale pastel shades (hue and saturation)
- Start the 9 colour programmes M1–M9 of the app and make them run faster or slower (S+ / S-)
- Control single zones of the 8 zones or all zones at once
- Connect everything with ioBroker scripts, schedules, visualisations and voice assistants

What the adapter cannot do (because the gateway does not report it):

- It cannot show which zone currently has which colour. The gateway knows only **one** state for all lamps: the last setting. The MiBoxer app does not show a difference between the zones either.
- It cannot show how fast a colour programme runs.
- It cannot see whether a lamp really received the radio command. It only sees what the gateway reports.

3. What you need

Tick off this list before you start:

- ☐ A **MiBoxer WL-433** gateway and at least one **PW01** or **PW02** lamp, set up in the **MiBoxer app** (chapter 4).
- ☐ A running **ioBroker** installation, for example on a Raspberry Pi. Minimum versions: **Admin 8**, **js-controller 6.0.11**, **Node.js 22**.
- ☐ Gateway and ioBroker in the **same home network**.
- ☐ The **device ID** and the **local key** of the gateway (chapter 5). For this you need an **Android phone or tablet** once – an iPhone or iPad alone is not enough (chapter 5 explains why).
- ☐ About 30 to 60 minutes of time.

TIP

How to check your ioBroker versions: open ioBroker Admin in the browser. In the menu on the left you find **Hosts** – it shows the versions of js-controller and Node.js. Under **Adapters**, the **Admin** tile shows the installed Admin version.

TIP

Fixed IP address for the gateway: set up a "DHCP reservation" for the gateway in your router (on a FRITZ!Box: *Home Network* → *Network* → *edit device* → *"Always assign this network device the same IPv4 address"*). Then its address never changes. This is not required, but it makes operation more reliable.

4. Step 1 – Set up the lamps and the gateway in the MiBoxer app

The adapter takes over the lamps the way they are set up in the app. So first set up everything in the app.

1. Install the **MiBoxer** app (manufacturer Futlight) on your phone – from Google Play (Android) or the App Store (iPhone/iPad).
2. Create an account and add the **WL-433 gateway**. Follow the instructions of the app.
3. Link every pool lamp to a **zone** of the gateway (zone 1 to 8). The guide [Anleitung_PW01_mit_WL-433_verbinden.pdf](#) (German) explains this step by step.
4. Try out in the app whether all lamps can be switched.

TIP

Write down which lamp is in which zone. You will need this later if you want to control single lamps:

Zone	Lamp (e.g. location)
1	
2	
3	

IMPORTANT

If you later delete the gateway from the app and add it again ("pair it again"), its **local key** changes. Then repeat chapter 5 and enter the new key in the adapter.

5. Step 2 – Get the device ID and the local key

The adapter needs two values to be allowed to talk to the gateway:

Value	What it looks like	What it is for
Device ID (<i>devId</i> in the log)	about 20 to 22 characters, e.g. <code>bf0123456789abcdefgh</code>	Tells the adapter <i>which</i> device is meant
Local key	exactly 16 characters, often with special characters, e.g. <code>a1B2\$c3D4{e5F6g7</code>	The "password" for the encrypted connection

The MiBoxer app does not show these values anywhere. But it writes them to its internal **log** – and this log can be read on an **Android device**.

The easiest way is the detailed guide [Anleitung_Geraete-ID_und_Local-Key_auslesen.md](#) (PDF, German). It explains every tap – completely without a computer using the apps *LogFox* and *Shizuku*, and also what you can do with an iPhone or iPad.

5.1 Alternative: with a computer and a USB cable (Windows)

This way was checked on 2026-09-22 with an Android 13 phone.

- 1. Unlock the developer options:** on the Android device open *Settings* → *About phone* and tap **Build number 7 times** until "You are now a developer" appears.
- 2. Switch on USB debugging:** switch on *Settings* → *System* → *Developer options* → *USB debugging*. On Xiaomi devices also switch on **USB debugging (Security settings)**.
- 3. Get the tool for the computer:** download the **SDK Platform-Tools** from Google: <https://developer.android.com/tools/releases/platform-tools>. Unpack the ZIP file, for example to `C:\sdk`.
- 4. Open a command prompt:** press the Windows key, type `cmd`, press Enter. Then change to the folder: `cd C:\sdk\platform-tools`
- 5. Connect the phone:** connect the phone with a USB cable and unlock it. It asks **"Allow USB debugging?"** – tick **"Always allow from this computer"** and tap **OK**.
- 6. Check the connection:** type `adb devices`. Your device must appear with the word `device`. If it says `unauthorized`, you have not yet confirmed the question from step 5: unlock the phone and look at the screen.
- 7. Start recording:** type `adb logcat -v time > miboxer.txt`. The window now seems to "hang" – that is correct, it is recording.
- 8. Use the app:** open the MiBoxer app, tap the gateway and switch the light off and on once.
- 9. Stop recording:** back at the computer, press **Ctrl + C**.
- 10. Search for the values:** type `findstr /i "localKey devId" miboxer.txt`. In the lines found, the device ID follows `devId` and the local key follows `localKey`.

CAUTION

The local key is like a password. Do not pass it on and do not publish the file `miboxer.txt` anywhere. Delete the file as soon as you have written down the values. Afterwards switch **USB debugging** off again in the developer options.

5.2 Only an iPhone or iPad available?

Apple does not allow any program to read the log of other apps. The easiest way is to borrow an Android device for ten minutes, install the MiBoxer app on it and log in with **your** MiBoxer account – the gateway then appears automatically, and you can proceed as described above. Further ways (Android emulator on the computer) are described in the detailed guide; they are **(unverified)**.

TIP

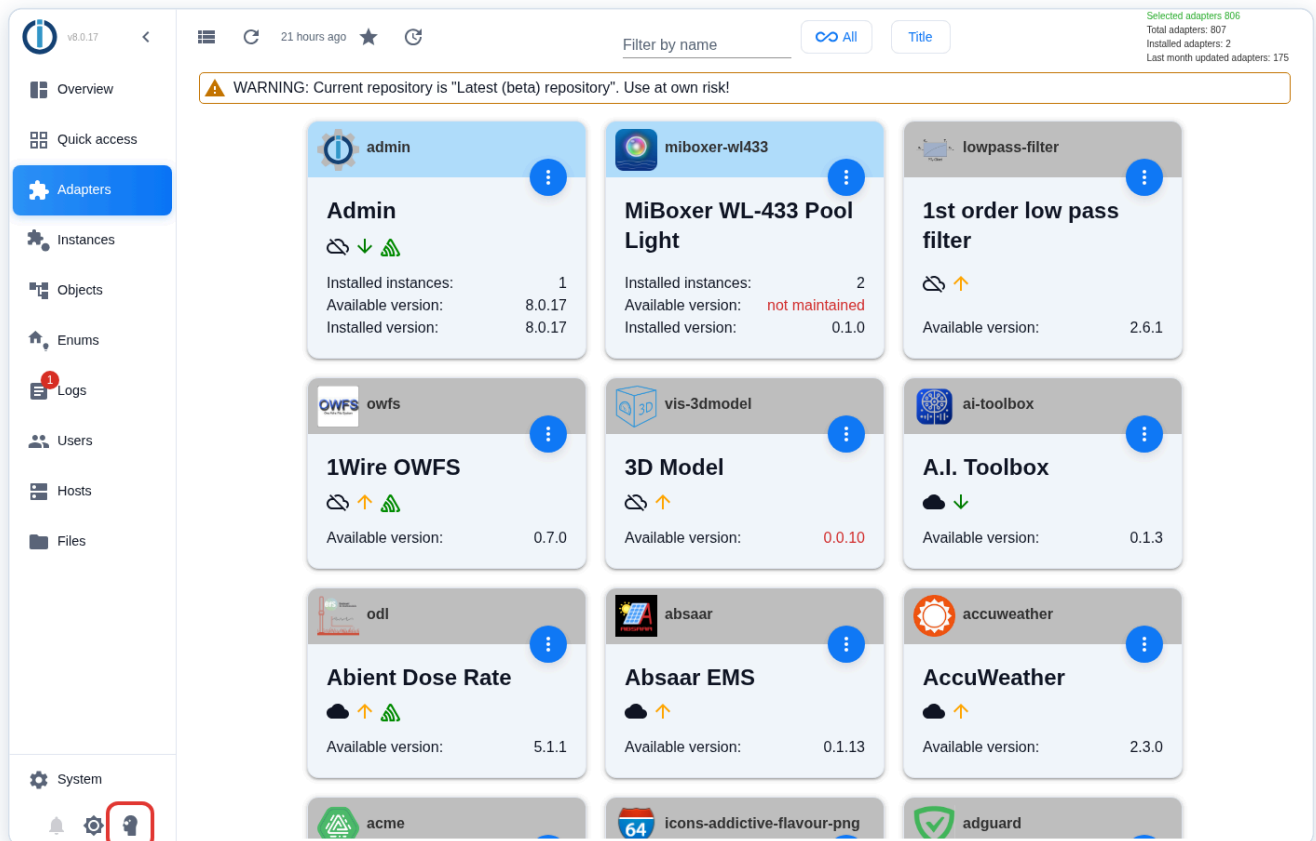
You can also find the **device ID** without a log: after installing the adapter, the button **Search gateway in the local network** (chapter 7) searches all Tuya devices in your home network and shows their IDs. It cannot find out the **local key**, though.

6. Step 3 – Install the adapter

The adapter is not yet in the official adapter list of ioBroker. So you install it directly from GitHub. As soon as it has been added to the list, it is enough to search for `miboxer` on the **Adapters** tab and click **+** (install).

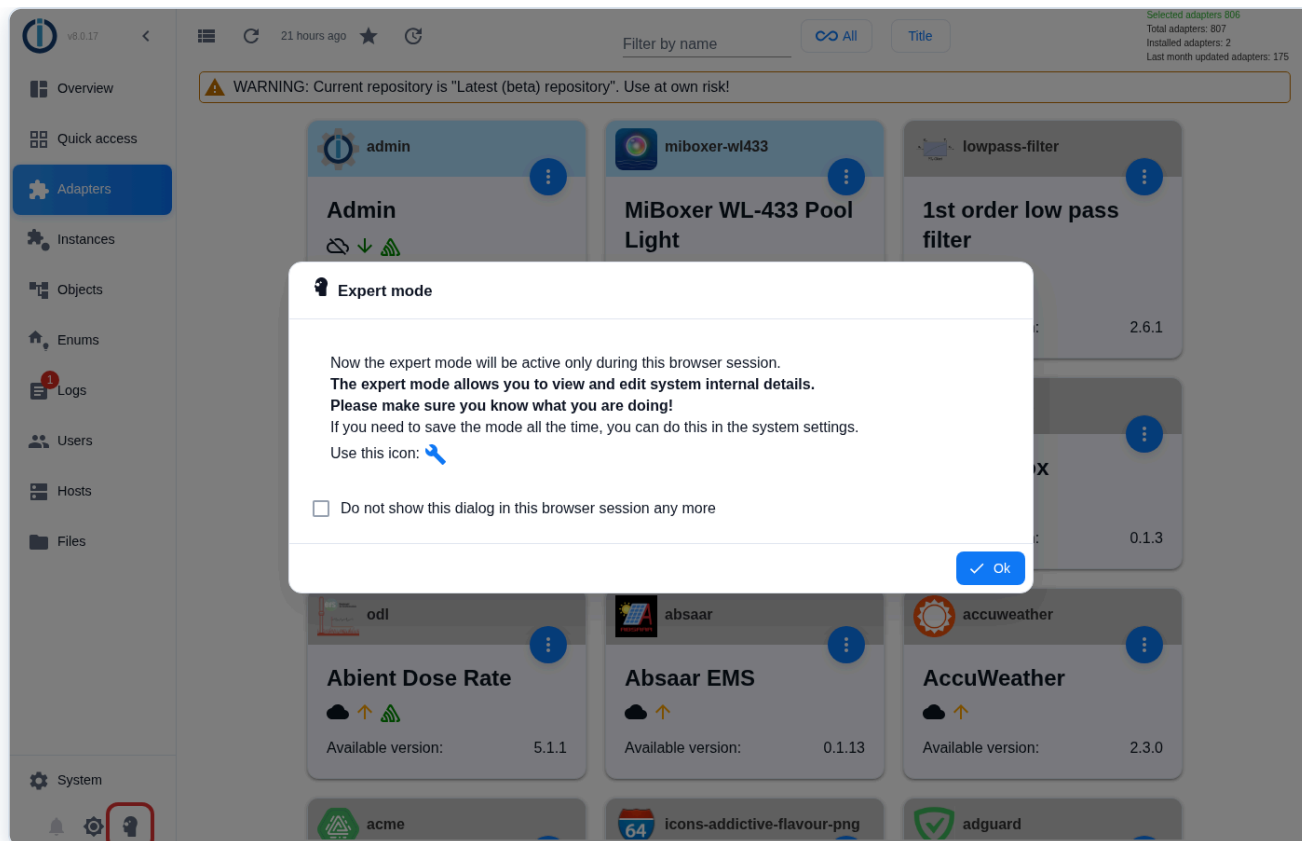
6.1 Switch on expert mode

1. Open ioBroker Admin in the browser, usually at `http://<IP address of your ioBroker>:8081`.
2. Click **Adapters** in the menu on the left.
3. Click the **head symbol** at the bottom left (marked in red).



▲ Switch on expert mode: head symbol at the bottom left

4. A note about expert mode appears. Click **Ok**.



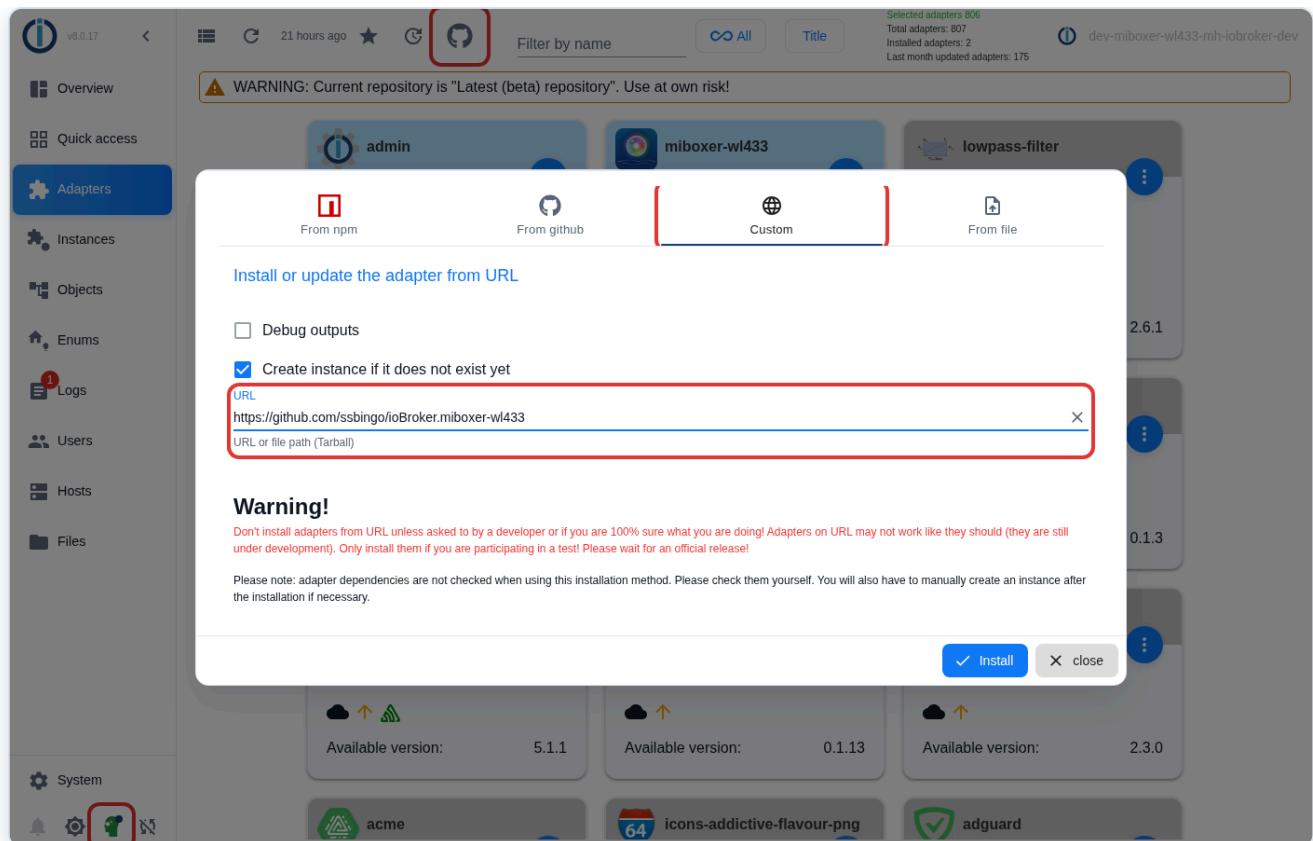
▲ Note about expert mode

NOTE

Expert mode shows additional buttons. It only applies to this browser session and does no harm. You can switch it off again with the same button.

6.2 Install from GitHub

1. A **cat symbol** (the GitHub logo) with the tooltip **Install from custom URL** now appears in the bar at the top. Click it.
2. Choose the tab **Custom** in the window.
3. Enter exactly this address in the field **URL**: `https://github.com/ssbingo/ioBroker.miboxer-wl433`
4. Keep the tick at **Create instance if it does not exist yet**.
5. Click **Install**.



▲ Install from custom URL: "Custom" tab and address

6. A window shows the progress. Wait until the installation has finished (on a Raspberry Pi this can take a few minutes) and close the window.

NOTE

The red warning in the window is a general note of ioBroker for all adapters that do not come from the official list.

Alternative for experienced users – on the console (for example via SSH on the ioBroker computer):

```
iobroker url https://github.com/ssbingo/ioBroker.miboxer-wl433
iobroker add miboxer-wl433
```

7. Step 4 – Set up the instance

After the installation there is an **instance** called `miboxer-wl433.0` – this is the "running adapter" for your gateway.

1. Click **Instances** on the left.
2. Find the row **miboxer-wl433.0** and click the **spanner symbol**. The settings page opens:

The screenshot shows the 'Instance settings: miboxer-wl433.1' page. The left sidebar has 'Instances' selected. The main content area has a blue header 'WL-433 gateway'. Below it, a note says 'The gateway is controlled locally via the Tuya LAN protocol. Device ID and local key can be read from the debug log of the MiBoxer app (see README)'. There's a guide link. The settings are organized into sections: 'Device ID' (bf000000000wl433abc), 'Local key' (16 characters), 'IP address of the gateway' (192.168.1.50), and 'Tuya protocol version' (3.3). A search bar is present. The 'Connection' section has 'Reconnect delay' (30s) and 'Status refresh interval' (60s). The 'Zones' section has a 'Zone selector (light.zone)' dropdown. At the bottom, there are 'Save', 'Save and close', and 'Close' buttons.

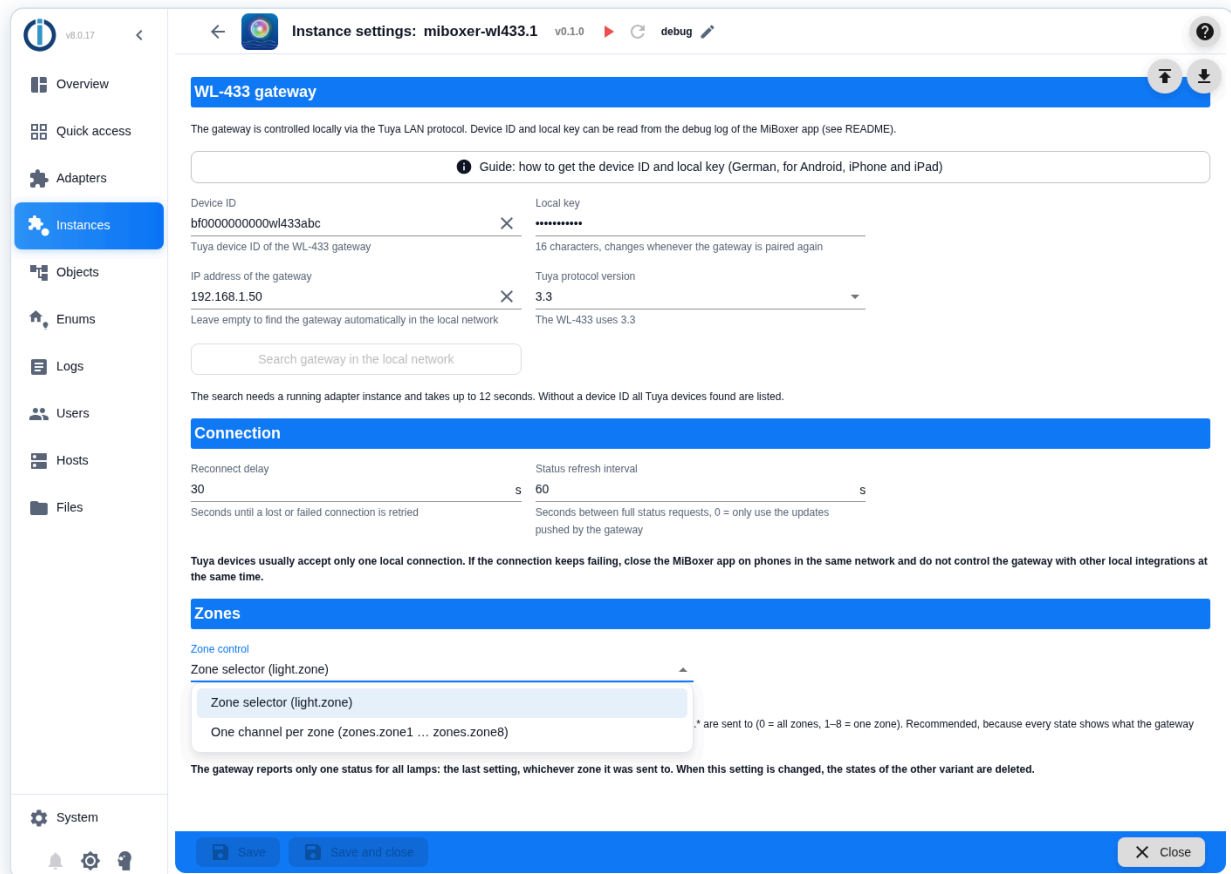
▲ Settings page of the instance (example values)

3. Fill in the fields:

Field	What you enter	Example
Device ID	The device ID from chapter 5	bf0123456789abcdefgh
Local key	The local key from chapter 5 – exactly 16 characters, mind upper and lower case	a1B2\$c3D4{e5F6g7
IP address of the gateway	The IP address of the gateway from your router. Or leave it empty – then the adapter searches the gateway itself	192.168.1.50
Tuya protocol version	Keep 3.3	3.3
Reconnect delay	After how many seconds the adapter tries again after a problem. 30 is fine	30

Status refresh interval	Every how many seconds the adapter asks for the state. <code>60</code> is fine	<code>60</code>
Zone control	How zones are offered – see below. If in doubt, keep Zone selector	Zone selector

4. Choose the zone control: click the selection field. There are two options:



▲ Selection of the zone control

Choice	What you get	Take this choice if ...
Zone selector (light.zone)	One set of states <code>light.*</code> and a state <code>light.zone</code> that selects the zone the commands go to	... you mostly control all lamps together or you are unsure. Recommended.
One channel per zone	In addition the folders <code>zones.zone1</code> to <code>zones.zone8</code> with their own states per zone	... you want to address single zones directly in scripts or visualisations

Chapter 10 explains both variants with examples.

5. Click **Save and close** at the bottom. The instance starts and connects to the gateway.

TIP

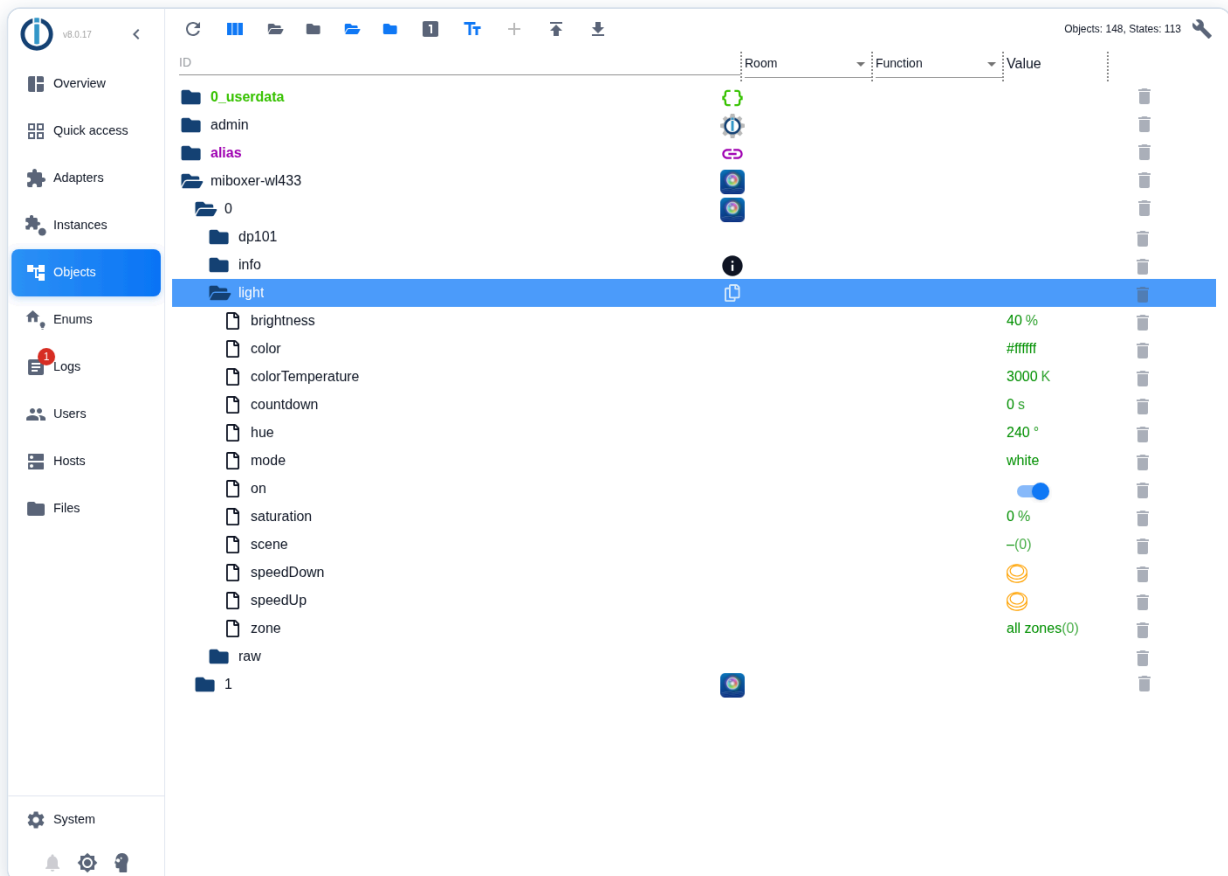
IP address unknown? Enter device ID and local key, click **Save** (do not close) and then **Search gateway in the local network**. The search takes up to 12 seconds. If it finds the gateway, it enters IP address and protocol version itself – afterwards click **Save and close** once more. The button only works while the instance is running.

IMPORTANT

If you change the **zone control** later, the adapter deletes the states of the other variant. Scripts that use these states then have to be adapted.

8. Step 5 – Check that everything works

- 1. Instance status:** click **Instances** on the left. The symbol at the beginning of the row **miboxer-wl433.0** shows the state – just as for all ioBroker adapters:
 - **green:** running and connected to the gateway – everything is fine;
 - **yellow:** running, but not (yet) connected – wait a moment, otherwise see chapter 12;
 - **red:** stopped – start it with the start button (triangle).
- 2. Look at the states:** click **Objects** on the left and open the folders **miboxer-wl433** → **0** → **light** one after the other (click the folder symbol). The values should match what the MiBoxer app shows:



▲ States under mixboxer-wl433.0.light

- 3. Check the connection:** in the folder **info**, the state **connection** must be `true`.
- 4. First switching test:** click the value in the row **brightness**, type `50` and press Enter. After about **2 to 3 seconds** the app also shows 50 % brightness.

NOTE

Why does it take 2 to 3 seconds? The gateway reports its new state only about 2.5 seconds after the last change. Only then the adapter "acknowledges" the value. Until then, Admin shows the entered value as not yet acknowledged.

If something does not work, continue reading in chapter 12.

9. Operating the pool lights

9.1 Changing a value

On the **Objects** tab you change a value like this:

1. Click the value of the state in the column **Value**.
2. Enter or select the new value. Switches (like **on**) are simply clicked, buttons (like **speedUp**) too.
3. Confirm with Enter or **Apply**.

In scripts, visualisations or voice assistants you use the same states – only automatically (chapter 11).

9.2 What do you want to do?

All states are in the folder `miboxer-wl433.0.light` :

You want to ...	State	Value (example)
switch on	<code>light.on</code>	<code>true</code> (switch on)
switch off	<code>light.on</code>	<code>false</code> (switch off) – or <code>light.brightness</code> to 0
make it brighter or darker	<code>light.brightness</code>	1 to 100 (percent)
warm white light	<code>light.colorTemperature</code>	2700
neutral white light	<code>light.colorTemperature</code>	4500
cold white light	<code>light.colorTemperature</code>	6500
a colour	<code>light.color</code>	<code>#0000ff</code> (blue) – or <code>light.hue</code> with a degree value
a paler colour	<code>light.saturation</code>	0 (white) to 100 (strong)
start a colour programme	<code>light.scene</code>	1 to 9 (M1–M9 in the app)
colour programme faster	<code>light.speedUp</code>	press the button
colour programme slower	<code>light.speedDown</code>	press the button
back to white light	<code>light.mode</code>	<code>white</code>
back to the last colour	<code>light.mode</code>	<code>colour</code>
toggle automatically after some time	<code>light.countdown</code>	seconds, e.g. 3600 for one hour

9.3 Colours

The easiest way to set colours is `light.color` in the format `#RRGGBB` , or `light.hue` as an angle on the colour wheel:

Colour	<code>light.hue</code>	<code>light.color</code>
Red	0	<code>#ff0000</code>
Orange	30	<code>#ff8000</code>
Yellow	60	<code>#ffff00</code>
Green	120	<code>#00ff00</code>
Turquoise	180	<code>#00ffff</code>
Blue	240	<code>#0000ff</code>
Violet	270	<code>#8000ff</code>
Pink	300	<code>#ff00ff</code>

The **brightness** is always set separately with `light.brightness`. A dark RGB value such as `#800000` therefore gives **strong red at the current brightness** – not a dark red.

9.4 What the adapter does automatically

- **Switching on when needed:** if you set brightness, colour, white tone or a programme while the light is off, the adapter first switches it on – just like the app.
- **Mode change:** a colour temperature automatically switches to white light, a colour to coloured light.
- **Sliders:** if you change a value very quickly in a row (for example with a slider), the adapter sends only the last value.
- **Confirmation:** every command only counts as executed when the gateway reports the new state (after about 2.5 seconds). If the report does not arrive, the adapter asks and writes a warning to the log (chapter 12).
- **Matching the app:** what you change in the MiBoxer app also appears in ioBroker. In addition, the adapter asks for the state regularly (setting *Status refresh interval*).

10. Zones

The gateway can control up to **8 zones** separately (zone 1 to 8) or all together ("ALL"). Which lamp belongs to which zone was set in the app in chapter 4.

IMPORTANT

The gateway reports **only one state for all lamps** – that of the last setting, whichever zone it was sent to. If you set zone 1 to red and then zone 2 to blue, `light.*` shows "blue", although the lamps in zone 1 are still red. This is a property of the gateway: the MiBoxer app does not show a difference between the zones either.

10.1 Variant "zone selector" (default)

With the state `light.zone` you choose where the commands of `light.*` go:

<code>light.zone</code>	Commands go to
0	all zones
1 to 8	only this zone

Example – only the lamps in zone 2 shall light up blue:

1. Set `light.zone` to 2.
2. Set `light.color` to `#0000ff`.
3. Afterwards set `light.zone` back to 0, so that later commands go to all zones again.

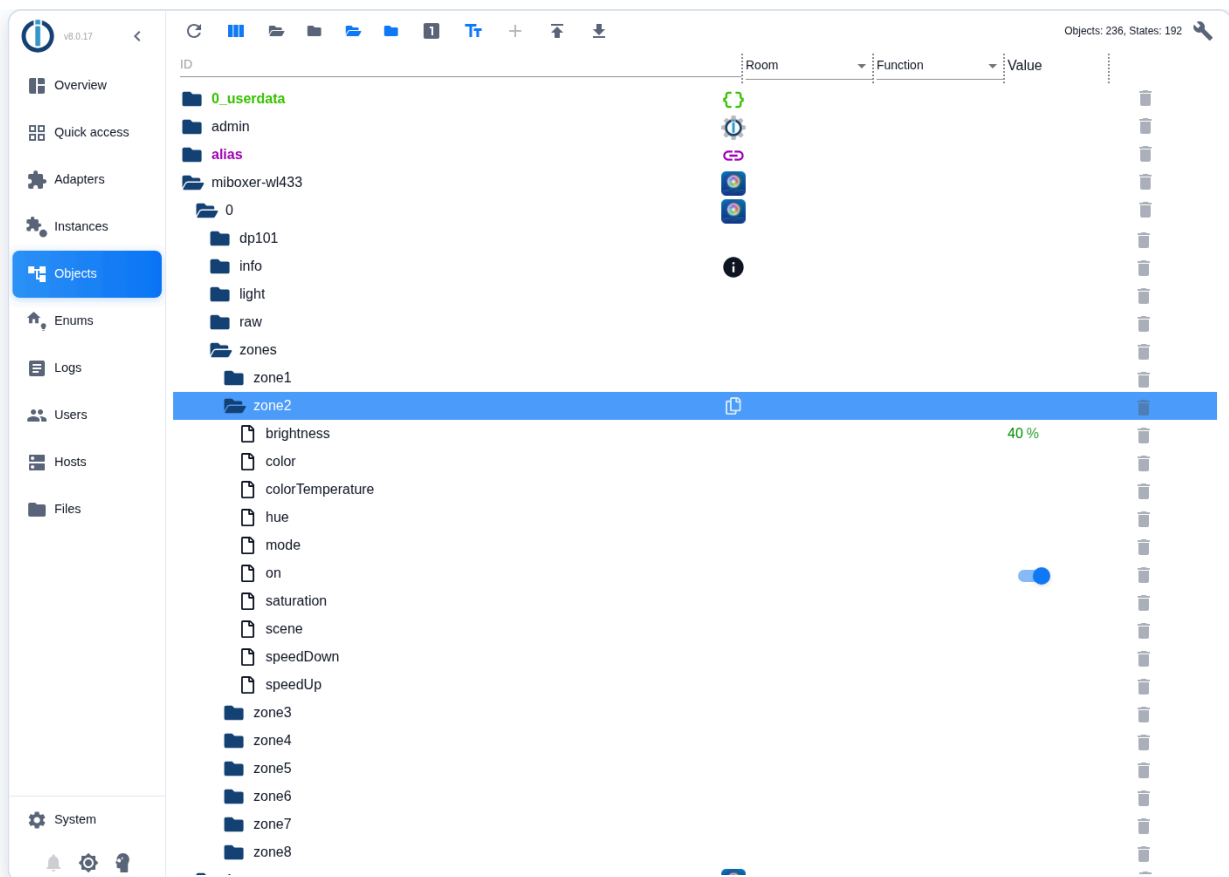
TIP

`light.zone` stays set until you change it – even after a restart. After targeted commands to a zone it is best to set it back to 0.

10.2 Variant "one channel per zone"

Here there is additionally the folder `zones` with one subfolder each `zone1` to `zone8`. Each has the same states as `light` (on, brightness, colorTemperature, color, hue, saturation, mode, scene, speedUp, speedDown). In this variant, `light.*` always sends to **all** zones.

Example – only zone 2 blue: set `zones.zone2.color` to `#0000ff`. Done.



▲ Zone channel zone2: only the confirmed values are filled in

NOTE

A zone channel shows the values **last sent to this zone and confirmed by the gateway**. Fields stay empty until you have sent something to the zone. In the picture, only "on" and "brightness 40 %" were sent to zone 2. A command via `light.*` (all zones) enters its value in all eight zone channels.

11. Automation examples

11.1 With scripts (JavaScript adapter)

For this, install the adapter **JavaScript/Blockly** (**Adapters** tab, search `javascript`). Create a new JavaScript under **Scripts** and insert:

```
// At sunset: switch the pool lights on in blue at 60 % brightness
schedule({ astro: "sunset" }, async () => {
  await setStateAsync("miboxer-wl433.0.light.color", "#0000ff");
  await setStateAsync("miboxer-wl433.0.light.brightness", 60);
});

// Switch off at 11 pm
schedule("0 23 * * *", async () => {
  await setStateAsync("miboxer-wl433.0.light.on", false);
});
```

Switch only one zone – variant "zone selector":

```
// Zone 2 to colour programme M3, then address all zones again
await setStateAsync("miboxer-wl433.0.light.zone", 2);
await setStateAsync("miboxer-wl433.0.light.scene", 3);
await setStateAsync("miboxer-wl433.0.light.zone", 0);
```

Switch only one zone – variant "one channel per zone":

```
await setStateAsync("miboxer-wl433.0.zones.zone2.scene", 3);
```

11.2 With Blockly (without programming)

Blockly is part of the **JavaScript/Blockly** adapter. You put building blocks together: **(unverified)**

1. Create a new **Blockly** script under **Scripts**.
2. Choose a trigger from the group **Timeouts / schedule**, for example **Astro** with *sunset*.
3. Drag the block **control** from the group **System**, select the object ID `miboxer-wl433.0.light.color` and enter `#0000ff` as the value.
4. Save and start the script.

11.3 Visualisation and voice assistants

The states have the "roles" common in ioBroker (for example *light switch*, *dimmer*, *colour RGB*, *colour temperature*). Visualisations and adapters for voice assistants can therefore usually recognise the pool lights automatically as a lamp. **(unverified)**

12. Troubleshooting

12.1 Switching on the detailed log

For troubleshooting, the adapter can write every step to the log:

1. Open the settings of the instance (chapter 7, spanner).
2. At the top, next to the name, you see the **log level** (for example `info`). Click the **pencil** next to it and choose `debug`.
3. You can also find the pencil in the picture in chapter 7 at the top, right of `v0.1.0`.

You see the log on the left under **Logs**. Type `miboxer` into the filter field at the top to see only the messages of this adapter. Every message starts with a tag in square brackets, for example `[conn]` for the connection or `[cmd]` for commands.

TIP

Set the level back to `info` after troubleshooting. The level `debug` produces a lot of lines.

12.2 Messages and what you can do

Message in the log (beginning)	Meaning	What you can do
<code>[cfg] Adapter is not configured: please enter the device ID and the local key ...</code>	Device ID or local key are missing	Chapter 7: fill in both fields
<code>[cfg] The local key must have exactly 16 characters ...</code>	The local key is too short or too long	Copy the key exactly (16 characters, no spaces)
<code>[conn] Cannot keep a connection to the WL-433 gateway ...</code>	The gateway does not let the adapter in	Check IP address, local key and protocol version; close the MiBoxer app on all phones; switch off other Tuya integrations (ioBroker.tuya, Home Assistant) for this gateway
<code>[rx] Gateway sent data that could not be decoded ...</code>	The local key is wrong or outdated (gateway paired again)	Repeat chapter 5 and enter the new local key
<code>[disc] Gateway ... did not announce itself in the local network ...</code>	The automatic search did not find the gateway	Read the IP address from the router and enter it; gateway and ioBroker must be in the same network
<code>[cmd] #12 light.brightness: the gateway did not confirm ...</code>	After a command, the gateway did not report a matching state	Is the gateway switched on and in the WiFi? Check in the app whether it is reachable. If it happens often: report a problem (12.4)
<code>[poll] ... Status query not answered ...</code>	The gateway does not answer the state query	Check that it really is a WL-433; report a problem (12.4)

<code>[cmd] ... not executed: a colour like "#ff8800" is expected</code>	The value entered has the wrong format	Enter the colour in the format <code>#RRGGBB</code>
<code>[cmd] ... not executed: gateway is not connected</code>	The command came while there was no connection	Wait until <code>info.connection</code> is <code>true</code> again and repeat the command
<code>[dp101] Status frame ... has an unknown mode ...</code>	The gateway reports an unknown state	Report a problem (12.4)
<code>[cfg] light.mode is created again without the mode "music" ...</code>	One-time change when updating from version 0.0.1	Nothing – set custom settings of this state (e.g. history) again if needed
<code>[cfg] Zone mode "...": removed ...</code>	After changing the zone control, the states of the other variant were deleted	Nothing – adapt scripts if needed

12.3 Frequently asked questions

All zones show the same colour, although the lamps shine differently. That is correct: the gateway reports only one state for all lamps (chapter 10).

A value jumps back briefly after changing it or is only taken over after 2 to 3 seconds. The gateway reports its state about 2.5 seconds after the last change. Only then the adapter takes over the value for good.

After pairing the gateway again in the app, nothing works any more. The local key has changed. Read it again (chapter 5) and enter it (chapter 7).

The IP address of the gateway has changed. Leave the field **IP address of the gateway** empty – then the adapter searches the gateway itself. Even better: set up a DHCP reservation in the router (chapter 3).

The speed of the colour programme is not shown anywhere. The gateway does not report it. `speedUp` and `speedDown` still work.

The adapter reports success, but a lamp does not react. The adapter only sees the gateway, not the lamps. Check whether the lamp is linked to a zone and whether it reacts in the app. Lamps under water have a shorter radio range.

12.4 Reporting a problem

Open an "issue" on GitHub: <https://github.com/ssbingo/ioBroker.miboxer-wl433/issues>. Helpful are:

1. the adapter version (shown at the top of the instance settings, e.g. `v0.1.0`);
2. a log at level **debug** – from the start of the instance to the error (12.1);
3. the content of the state `dp101.history` (click it on the Objects tab under `miboxer-wl433.0.dp101` and copy it);
4. a short description of what you did and what happened.

NOTE

The adapter **never** writes the local key to the log – the log only shows its length. So you can attach the log without worry. But do **not** attach recordings of the MiBoxer app (chapter 5), because they contain the key.

13.3 Reproducing the protocol yourself

The format was decoded like this on 2026-09-22 – you can check it with the same method or extend it for a newer gateway firmware:

- 1. Read the commands of the app:** connect an Android device to the computer as in chapter 5.1 and start `adb logcat -v time > commands.txt`. The MiBoxer app writes every command as a line `ayxsendData =<hex values>` to the log.
- 2. Exactly one action per step:** carry out exactly **one** action in the app (for example "brightness to 50 %"), write down the time and wait about 10 seconds until the next action.
- 3. Find the commands:** `findstr "ayxsendData" commands.txt` (Windows) or `grep ayxsendData commands.txt` (Linux, macOS) lists all commands sent.
- 4. Compare the states:** the state `dp101.history` of the adapter shows the matching state reports of the gateway with their time.
- 5. Send yourself:** in `dp101.hex` you can enter 11 bytes – the adapter adds the checksum and sends the frame. Example: `43 00 00 80 00 00 00 00 00 80 80` asks for the state.

CAUTION

The file `commands.txt` also contains the local key. Do not publish it – publish only the `ayxsendData` lines you picked out.

The complete derivation with all evidence is in the protocol analysis (German), chapter 3.1.8: [Miboxer_WL-433_PW01_Protokollanalyse_lokale_Steuerung.md](#).

14. Glossary

Term	Explanation
Adapter	An add-on program for ioBroker that connects a certain device or service
Instance	A running adapter with its own settings, e.g. <code>miboxer-wl433.0</code>
State	A single value in ioBroker, e.g. <code>light.brightness</code> . Visible on the Objects tab
Acknowledged (ack)	The value was reported or confirmed by the device – not just entered
Gateway	The WL-433 device that mediates between WiFi and radio
Zone	A group of lamps that are switched together (1 to 8)
Scene / colour programme	The programmes M1–M9 of the app with changing colours
Device ID	The unique identifier of the gateway at Tuya
Local key	The secret 16-character key for the encrypted connection in the home network
Tuya	Manufacturer of the WiFi module in the gateway and of the related cloud
LoRa	The radio technology the gateway uses to reach the lamps (433 MHz)
Log	A diary in which a program writes what it does
adb	A tool from Google that lets a computer access an Android device
DHCP reservation	A router setting that always gives a device the same IP address

15. Legal notice and contact

This is an **unofficial community project**. It is not affiliated with Shenzhen Futlight Optoelectronics Co., Ltd. (MiBoxer / Mi-Light) or Tuya. "MiBoxer", "Mi-Light" and "Tuya" are trademarks of their respective owners and are only used to describe compatibility. Use at your own risk.

- Source code and questions: <https://github.com/ssbingo/ioBroker.miboxer-wl433>
- License: MIT – Copyright (c) 2026 ssbingo